# How to Set Up Laya (Step-by-Step Guide)

Oct 25, 2026 · @Aditya Anand

## What Laya is

Laya is a free, open-source decision model you run on your own machine. It does the same job as TypeSafe's Jev: you give it some text and a list of allowed answers, and it picks one with a probability attached.

It reads the question once and answers in a single forward pass. It does not generate text, so there is nothing to parse. Laya is built by Nandakishor M at Convai Innovations.

- **Size:** 421M parameters for the English checkpoint (ModernBERT-large)
- **Speed:** about 33 ms per question on a T4 GPU, measured by the Laya project; third-party tests put Jev at 236–276 ms
- **License:** Apache 2.0, with open weights you can download and fine-tune
- **Catch:** the base model is not good at everything out of the box, so you fine-tune it for your own use case (Step 4)

## What you need

A laptop with Python 3.10 or newer is enough to run Laya; a GPU makes it faster, and fine-tuning needs a few hundred labeled examples.

- **Python 3.10 or newer** and `pip` (or `uv`)
- **Internet on first run:** the checkpoint downloads from Hugging Face the first time you use it, then it is cached
- **Hardware:** CUDA GPU, Apple Silicon, Intel GPU or plain CPU. Laya picks automatically in this order: CUDA, MPS, XPU, CPU
- **Disk:** the three checkpoints total about 2.3 GB; the default `Router()` downloads only the one it needs
- **For Step 4:** Claude Code in your terminal, plus a few hundred examples of the decision you want, with the correct answer attached

Speed depends on your hardware. The ~33 ms figure is a T4 GPU measurement from the Laya README; the same README lists 193–464 ms on CPU with models preloaded.

# Step 1: Install Laya

Create a virtual environment, install Laya from PyPI, and run the version check. The check confirms the install without loading a checkpoint.

**macOS / Linux**

```
python3 -m venv .venv
.venv/bin/python -m pip install laya
.venv/bin/python -I -c "import laya; print(laya.__version__)"
```

**Windows PowerShell** (example uses an installed Python 3.11)

```
py -3.11 -m venv .venv
.\.venv\Scripts\python.exe -m pip install laya
.\.venv\Scripts\python.exe -I -c "import laya; print(laya.__version__)"
```

**Using uv** (same on every platform)

```
uv venv --python 3.12
uv pip install laya
```

If something fails:

- `ModuleNotFoundError: No module named 'laya'`: run your script with the same virtual environment's Python that you installed into.
- On Debian or Ubuntu, `venv` may need `sudo apt install python3-venv` first.
- On an Intel Mac, a plain install resolves to a stack that cannot run. The Laya README says to pin it: `pip install "numpy<2" "torch==2.2.2" "transformers==4.57.6" laya`.

# Step 2: Run your first decision

Try the command line first, then the Python call. Both download the checkpoint from Hugging Face the first time.

**Command line**

```
laya "My payment failed twice" --preset triage
```

`--preset` answers a ready-made question set. The presets are triage, email, guard, moderation and router.

**Python**

```
from laya import Router

router = Router()  # downloads only the selected checkpoint on first use
result = router.predict(
    {"body": "We were billed twice. Please refund the duplicate."},
    {"billing": {"type": "noul", "instructions": "Does the user request a
refund?"}},
)
print(result["answers"]["billing"]["noul"])  # P(true), a float from 0.0 to
1.0
print(result["routing"]["model"])            # which checkpoint answered
```

You give Laya a state (any text, email, ticket or JSON) and a dict of typed questions. There are three question types:

| Type | You get back | Use it for |
|---|---|---|
| `choice` | the top label plus a probability per option | department routing, intent, topic |
| `score` | expected level on an ordered scale | urgency, frustration, severity |
| `noul` | probability that a yes/no statement is true | spam, phishing, churn risk |

The `Router` picks the English or multilingual checkpoint for each request. Text it cannot place goes to the multilingual checkpoint by default; if most of your traffic is English, use `Router(default="english")`.

## Step 3: Serve Laya locally (optional)

Run `laya-serve` to get a local HTTP server that speaks the same `POST /v1/systemone` protocol as TypeSafe's hosted Jev API. An existing Jev client only needs its base URL changed.

```
pip install "laya[serve]"
LAYA_HOST=127.0.0.1 LAYA_MODELS=english laya-serve
```

Binding to `127.0.0.1` keeps the server reachable from your own machine only. Then send it a request from another terminal:

```
curl -s localhost:8000/v1/systemone -H 'content-type: application/json' -d '{
  "state": {"body": "billed twice, refund please or we cancel"},
  "questions": {"dept": {"type": "choice", "instructions": "which team?",
                "criteria": {"billing": "refunds", "tech": "bugs"}}}
}'
```

Useful settings, all environment variables:

- `LAYA_DEVICE`: pin `cuda`, `mps` or `cpu`
- `LAYA_PRELOAD=1`: load the checkpoints at startup instead of on the first request
- `LAYA_API_KEY`: when set, clients must send `Authorization: Bearer <key>`

## Step 4: Fine-tune Laya with Claude Code

Fine-tuning is where most of Laya's accuracy comes from. On the project's typed-decisions benchmark, the base English checkpoint scores 0.362 and the fine-tuned checkpoint scores 0.766.

1. **Pick one decision.** For example: which team handles this ticket, is this spam, how urgent is this.
2. **Collect a few hundred examples** of that decision with the correct answer attached. Check the labels yourself, because bad labels give bad decisions.
3. **Open Claude Code in your terminal** and paste the prompt below.
4. **Answer its question** about the decision you want. It will then fine-tune, calibrate and write a results file.
5. **Read the results file** before you trust the model: accuracy before and after, and whether the confidence numbers hold up.

```
Become the expert on Laya (github.com/NandhaKishorM/laya) before changing
anything. Clone the repo, read the fine-tuning and calibration docs, and test
the base model on my data to see what it gets right and wrong.

Then ask me which decision I want it to make. Do not guess.

Once I answer: fine-tune Laya on my labeled examples, calibrate its
confidence so the numbers mean what they say, and keep going until it is
reliable at that one decision.

Keep a results file as you go: accuracy before and after, how long training
took, and whether the confidence numbers can be trusted. If something in the
docs does not hold up in a real test, tell me instead of working around it.
```

The prompt follows the approach in this walkthrough by Timothy Biondollo: give Claude a mission instead of a rigid checklist. He reports that in his tests about 200 examples got most of the way to what 1,000 did.

If you would rather run the training yourself, the Laya repository ships a Kaggle notebook for free 2xT4 GPUs and an Apple Silicon script. The README gives roughly 4–5 hours on 2xT4 for 4 epochs over about 30k questions. In Biondollo's laptop test, a fine-tune on

1,000 labeled reviews took under five minutes.

## Step 5: Calibrate and test

Laya's shipped checkpoints are over-confident, so fit calibration after fine-tuning and test on examples the model never saw. In the Laya README, refitting temperatures on held-out data cut mean calibration error from 0.466 to 0.081 for the English checkpoint.

1. **Keep a held-out set.** Set aside examples that were not used for training or calibration.
2. **Compare before and after** on that set. The results file from Step 4 should already show this.
3. **Gate on confidence.** Let the model act on high-confidence answers and send the rest to a person.

```
res = router.predict(state, questions, min_confidence=0.85)
ans = res["answers"]["department"]

if ans.get("low_confidence"):
    send_to_human(ans["choice"])
else:
    route_automatically(ans["choice"])
```

The 0.85 above is only an example. Pick your own threshold from measured accuracy on your held-out set; the README says a threshold is a policy you choose, not a property of the model.

To score a labeled file and fail a build when quality drops, use the built-in evaluation command:

```
laya-evals run data.jsonl --model english --min-accuracy 0.8 --max-ece 0.05
```

## Limits to know before you rely on it

Laya is a fast base to specialize, not a zero-shot decision engine, and the project's own README says so.

- **Zero-shot is weak on some tasks.** On the typed-decisions benchmark the base checkpoints score 0.362 and 0.352, below a 0.461 majority-class baseline. Fine-tune before you trust it.
- **Avoid yes/no or true/false as `choice` labels.** The model can follow the label words instead of the option descriptions. Use meaningful labels or `A` / `B`.

- **Many options hurt accuracy.** On a 77-label test, Laya scored 0.425 against Jev's 0.870. For more than about 20 options, raise the option budget or shortlist the labels first.
- **Check `noul` yes/no answers on your own data.** On the English checkpoint the label pair can dominate the answer. The README documents a `labels` override and a two-option `choice` as workarounds.
- **Non-English text needs the right checkpoint.** The English checkpoint fails on non-Latin scripts, which is why `Router` picks a checkpoint per request.
- **First call is slow.** The checkpoint downloads once. Use `Router(preload=True)` in a server so later calls stay fast.
- **Jev still leads in places.** The README lists high-cardinality label sets and raw out-of-the-box calibration as areas where Jev is ahead.

## Links and sources

This guide was written against Laya 0.4.15 (PyPI, checked October 25, 2026). Laya ships updates often, so run `pip install -U laya` and check the docs if a command behaves differently.

- Laya on PyPI: install, quickstart, server, fine-tuning and benchmark details (the main source for this guide)
- Laya GitHub repository
- Laya documentation
- Laya model on Hugging Face (download from the convaiinnovations organization)
- Live demo on Hugging Face Spaces
- How to Fine-Tune Your Own Open Source Jev Alternative by Timothy Biondollo: the Claude Code approach in Step 4